ABSTRACT:

This research proposal involves designing an efficient and scalable framework that can successfully differentiate various acoustic scenes from each other, for example, urban environment sounds, public spaces, sounds inside office spaces, and others. To achieve this objective, an extensive use of state-of-the-art machine learning methods and architectures will be made in order to develop a robust classifier capable of recognizing diverse acoustic scenes. This task will include the development of a well-balanced audio dataset, implementation of advanced signal processing techniques and models that enable the creation of relevant features, as well as designing neural models that are able to provide reliable classification results in real-time settings. One of the most critical issues that should be resolved in this project is the problem related to the non-stationary nature of noise signals and overlapping sound events that can distort classification performance. Therefore, this project should aim at improving the performance of state-of-the-art classification frameworks through the implementation of advanced techniques and methods.

INTRODUCTION

The field of automatic audio scene classification is quickly evolving into one of the most promising areas of study, due to its extensive applications in many areas. Being able to classify audio scenes would allow for the extraction of contextual information about acoustic environments, providing useful data for such fields as urban infrastructure management, intelligent surveillance, context-aware computing, and smart devices. With sensors becoming an increasingly prevalent part of modern urban landscapes, a reliable method for processing auditory signals becomes necessary. The proposed project will follow a systematic, scientifically sound methodology in creating an AI-powered audio scene classification system. It consists of several stages that include the creation of a high-quality, diverse dataset; creation of powerful signal preprocessing techniques; and training of ML and DL algorithms that are capable of learning to discriminate between different acoustic phenomena. This project also includes a focus on real-time audio classification, considering the associated problems of noise tolerance and processing speed.

PROBLEMOVERVIEW:

The Acoustic Scene Classification (ASC) problem refers to automatic prediction of the environmental context of a short audio recording via assigning a semantic label to it. In contrast to sound event classification, where each particular sound is detected separately, ASC tries to find the right semantic label based on the overall recording, indicating the presence of airport, park, bus, metro station, shopping mall, and other typical urban environments. Thus, ASC

combines various acoustic characteristics such as background noise, reverberations, and overlapping sound textures into one scene context.

In this work, ASC is approached with the use of TAU 2022 Urban Acoustic Scenes dataset containing recordings for 10 different types of scenes: airport, bus, metro, metro station, park, public square, shopping mall, street pedestrian, street traffic, and tram. For the development of the classification pipeline, a lightweight CNN architecture was chosen with feature learning on log-mel spectrograms enhanced with delta and delta-delta coefficients. Moreover, for the robustness purposes, the training process makes use of several state-of-the-art augmentations such as Spec Augment and Mix-up methods.

1.1Background and motivation

Audio Scene Classification (ASC) refers to a branch of Computational Auditory Scene Analysis (CASA), concerned with the detection, identification, and classification of soundscape environments through audio signals. The topic has undergone significant expansion due to its applicability to smart city infrastructures, assistive devices, security and surveillance systems, as well as large-scale environmental monitoring tasks.

Classical ASC Approaches

In the early years of ASC research, researchers tended to combine manually designed acoustic features with classical machine-learning classifiers. Methods, such as Gaussian Mixture Models (GMMs) and Hidden Markov Models (HMMs), were prevalent. They were highly dependent on the feature design process which needed a lot of experience to accomplish. Additionally, the manual feature design procedure took considerable time, often introducing human bias into the process.

New Advances in Machine Learning

With new advances in machine learning, however, some promising algorithms emerged. For instance, the Support Vector Machine (SVM) classifier showed superior performance, especially in cases where the number of features was relatively high and there was a shortage of training data. SVM provided an optimal combination between algorithm simplicity and classification accuracy.

Revolution by Deep Learning

ASC has undergone a revolution through deep learning due to the capacity of deep models to automatically learn discriminatory representations based on audio input data or spectograms. CNNs and RNNs perform well in detecting spatial and temporal structures in audio data without necessitating any feature engineering. Deep models have always outperformed other approaches due to their capability to learn hierarchical, task-specific features.

Current Issues and Opportunities

Nevertheless, there are still many issues that remain unsolved in ASC. Variability caused by factors such as background noise, environmental variations, or different devices can greatly affect model accuracy. Additional challenges arise when dealing with real-time scenarios involving processing limitations. However, current trends indicate that new opportunities are available for improvement in terms of preprocessing techniques, noise-resilient feature extraction, and neural architectures.

ASC has numerous applications. Smart devices can alter their behavior according to the acoustic environment, such as hearing aids in restaurants that can automatically switch to a noisy setting or mobile voice assistants that act differently in libraries. ASC is also valuable for robotics when used alongside visual sensors, especially in low light or occluded scenarios. ASC can be used to classify audio files in an audio archive or audio indexing.

Practical implementation presents some challenges, however. Feature-based approaches, although understandable, are less robust than deep neural networks. On the other hand, while DNN models outperform traditional methods in terms of accuracy, they need large amounts of data, elaborate preprocessing steps, and efficient architectures to run in resource-limited settings.

Audio Scene Classification using TAU (DCASE) Dataset

The research utilizes the TAU Urban Acoustic Scenes 2022 dataset, which is accessible through Zenodo as part of the DCASE 2022 Challenge Dataset Pack. This dataset features binaural audio clips belonging to 10 classes of urban acoustic scenes, such as:

Airport

Shopping mall

Metro station

Pedestrian street

Public square

Street with traffic

Tram

Bus

Metro

Urban Park

1. What is TAU

The TAU Urban Acoustic Scenes dataset is part of the DCASE challenge, which is a widely used research benchmark in audio machine learning. The task is to classify short audio clips into categories such as airport, park, street, bus, metro, and similar environments. Each sample is only one second long, and the goal is to determine the scene based on that very limited information. This is not a toy dataset. It is designed to test real-world robustness and generalization.

2. Why this is a difficult problem

There are several factors that make this task challenging.

Short duration

Each audio clip is only one second long. This means:

· very little temporal context

· fewer distinguishing features

· more ambiguity between classes

2. Data and Preprocessing

2.1 Dataset Splits

TAU 2022 comes with an officially specified fold-based split, which is followed by this work when dividing the dataset into subsets. By means of the provided fold files, the dataset is split into three subsets, each containing a fixed number of samples:

Training Set:

- Consists of 139,620 audio clips.
- Utilized to train the model, using all forms of augmentation (SpecAugment, Mix-up, etc.).
- Accounts for approximately 70% of the total data employed by the pipeline.
- Key Function: Weights optimization and scene-specific acoustic pattern learning.

Validation Set (Evaluation Set):

- Consists of 29,680 samples.
- Used for monitoring during training, tuning hyperparameters, and avoiding overfitting.
- Comprises about 20% of the dataset in terms of effective usage.

Test Set:

- Comprises 29,680 examples (same as the evaluation set).
- There is no separate test set besides the existing folds.
- Accounts for about 10% of the data, as per the desired 70/20/10 distribution.
- Due to its use from the validation set, the test accuracy mentioned is validation accuracy.
- This is a potential weakness in methodology: there may be some bias.

Since the evaluation split works as two splits simultaneously—one being used for tuning models and another being used for reporting results, this test accuracy denotes a performance score on the same split used for selecting the best performing model. It is necessary to remember this methodology flaw, and interpret the results accordingly, as performance scores on unseen data. Going one step further, this may result in overly optimistic performance figures since the model will be adapting to features of the evaluation split during its development phase. To properly benchmark models, future research needs to utilize an independent test set or employ cross-validation on all official folds.

2.2 Audio Preprocessing

All audio samples will go through a pre-processing pipeline for reasons of consistency before extracting features from the data. Pre-processing techniques applied include the following:

• Sampling Rate

All samples are loaded and re-sampled so that the consistent sampling rate will be 44,100Hz (44.1kHz).

This makes sure that all the samples have a consistent time resolution and are ready to be used by other techniques for feature extraction such as log mel spectrograms.

• Clip Length

The length of all clips will be made equal to the standard 1 second (44,100 samples).

In case a clip is less than the required standard, zeros will be added to make it meet the standard length. In case a clip is greater than the required length, then excess lengths will be cut.

This will ensure that each sample has the same number of features as input.

• Consistent Processing Across Splits

The exact same preprocessing steps are used for all datasets - training, validation, and testing.

It avoids any inconsistencies between distributions and guarantees that identical inputs are fed into the model both during training and inference.

• Motivation and Effect

Normalization of duration and sampling rate removes all irrelevant variations other than those specific to the acoustic scene being classified.

Through zero-padding and cropping, the model will not pay any attention to the clip length but will focus solely on its content.

2.3 Feature Extraction

Each second of the waveform is converted to the log mel spectrogram using 128 mel filters, which makes it perceptually relevant representation. Such transformation takes place in terms of the Short-Time Fourier Transform (STFT) applied with the Hann Window. It helps to provide smoothing of frame transitions and makes energy estimation along the timeline stable. Mel scale conversion compresses the frequency range in a way that better reflects human perception, thus becoming suitable for acoustic scenes analysis.

To account for temporal aspects of sound signals, there are used the following features:

- Delta (first derivatives) - measuring rate of change of energy.
- Delta-Delta (second derivatives) - measuring acceleration (curvature) of energy.
- Both features supplement static spectral representations like log mel spectrograms with information about the dynamics of the spectra.

The three resultant feature maps (log-mel, delta and delta-delta) are concatenated into a tensor along the channel axis to form a three-channel input like how an image has its RGB channels. This provides a mechanism for the CNN to process and analyze different properties like spectral magnitude, velocity, and acceleration at the same time, thereby enabling more efficient pattern recognition across various acoustic environments.

For efficiency and consistency during training, spectrograms corresponding to evaluation and testing sets are pre-computed prior to training and saved to disk. This way, computation is not repeated throughout the epoch of training and ensures that there is no inconsistency across experiments in terms of the training inputs used. This is important for consistency during validation and testing phases.

2.4 Normalization

All the input spectrograms are normalized by their per-sample, per-channel z-scores to provide consistency in the scale of all inputs. For every sample, mean and standard deviation are separately calculated for the log-mel, delta, and delta-delta channels. Then, the meaning of the channel is subtracted, followed by the division by the standard deviation of the same channel, giving a normalized output with mean zero and unit variance.

Normalization is done at inference time by using the statistics of the sample alone, without the need for statistics from other samples of the whole dataset. In that way, the model is robust against any changes in the recording amplitude, microphone sensitivity, or even different gains between devices. This minimizes loudness variance and thus stabilizes the trained CNN filters, allowing the network to pay more attention to relative spectral features rather than to magnitude. Moreover, the use of per-sample normalization guarantees that the dynamic range of each sample becomes equal, which helps in keeping the process numerically stable and avoids some channels from taking over the entire feature domain based on their larger scales. It should be noted that such normalization techniques are especially useful for acoustic scene classification applications.

3. Model Architecture

3.1 Overview

Classifier is built using Convolutional Neural Network architecture with the intention of minimizing its computational overhead while maximizing representational power. The classifier is a 3-channel log-mel spectrogram, having shape (3, Frequency, Time). It passes through a series of convolutional blocks in the CNN that gradually decrease its spatial dimensions and increase feature map channels until it outputs a score vector over all 10 acoustic scenes.

The goal of efficient model implementation comes from the choice of convolution operations used. Depthwise separable convolutions replace normal convolutions everywhere in the model and thus make it computationally less expensive than its counterpart of equal depth (in terms of parameters and floating-point operations required). This allows for the successful use of the network even in computationally restricted environments.

Despite its light weight, the model still exhibits high levels of representational ability, allowing the network to learn the discriminative features of different urban soundscape scenes through time and frequency spectra. The efficient nature of the model helps it achieve strong results in the TAU benchmark despite being implementable in situations

with resource constraints.

3.2 Key Components

Depth wise Separable Convolutions

Every convolutional block in the architecture employs depthwise separable convolutions as opposed to regular 2D convolutions. This procedure involves breaking down a conventional convolution layer to two operations:

- **Depth wise convolution** applies the same spatial filter for each individual input channel, allowing the model to learn spatial patterns for each separate channel.
- **Pointwise (1x1) convolution** merges the results obtained in the previous layer, making the features from different channels interact.

As one can see, this approach helps tremendously in terms of computation. Namely, compared to a regular 3x3 convolution layer, the number of required computations is 8-9 times less, while losing almost none of its representation ability.

In addition to reducing parameters, this architecture design is also able to optimize memory usage, reduce latency, and make the architecture deployable even in embedded environments. Although relatively simple, the use of depth wise and pointwise convolution helps to capture complex spectral-temporal information, thereby making the architecture capable of performing well on the TAU acoustic scene classification dataset.

Batch Normalization

Batch normalization (BatchNorm2d) is used in each convolutional layer after the convolutional operation but before applying the activation function. This step ensures that the feature maps produced by the convolutional layers are normalized based on their mean and variance within each mini batch such that the resulting mean value is 0 and the standard deviation is 1. The benefit of using this approach is the elimination of internal covariate shift, thus facilitating faster learning.

Batch normalization offers an added advantage through the regularization effect it introduces to the training process. It improves the robustness of training by making the model less sensitive to the choice of hyperparameters such as learning rate and the method for initializing the weights. Moreover, it prevents overfitting without the need to add other types of regularizes.

Activations and Pooling

Rectified Linear Units (ReLU) functions are used right after the application of each batch normalization stage in the convolutional blocks. It is the combination of the two that makes up for a non-linearity which enables the network to learn the non-linear relations in the input data. The properties of the function enable the network to have sparse activations and to accelerate learning process by focusing on extracting the most informative features.

Max pooling is the next step performed on top of the convolutions. Max pooling is the step of down sampling and reducing the number of parameters to be learned while at the same time increasing the receptive field of the filter. As already mentioned, increasing the receptive field of the network enables deeper layers to understand more complex features of the audio/visual stream. In addition to that, max pooling contributes to achieving translation invariance as the output will remain the same no matter where the relevant pattern appears in the image.

Global Average Pooling and Fully Connected Layers

Following the last convolutional block, the network makes use of Global Average Pooling (GAP) to obtain a single representative scalar value for each of the feature maps through averaging of the respective activations across all spatial dimensions. In this way, multidimensional feature maps get compressed to a fixed-size, low-dimensional embedding vector that does not depend on the spatial dimension of the input anymore. When compared to a standard flattening layer, which would result in a highly dimensional vector and hence, in an increase in the number of trainable parameters, this approach presents a much more efficient way to achieve similar results.

Once passed through Global Average Pooling, the obtained GAP vector serves as an input for one or several fully connected (or linear) layers, where a ReLU activation function is commonly applied to allow the network to learn more sophisticated decision boundaries in the latent space and improve its expressive capabilities. After being processed through the densely connected layers, the high-level representation of the signal acquired from convolutional layers gets refined further. The neural network ends with a linear layer with ten output units, each one representing a particular acoustic scene.

3.3 Input/Output Dimensions

The architecture takes into account the tensor inputs with dimensions (Batch, 3, F, T) where F indicates the number of mel-frequency bins (equal to 128 in this implementation) and T is the number of time frames that are derived from one second of audio depending on the selected hop size. The three different channels are used to represent the log-mel spectrogram, the first order delta and second order delta-delta, allowing for the capturing of the time dynamics together with the spectral features of the audio clip.

Following the forward pass through the backbone convolutional network, pool layers, and fully connected classifier head, the model will generate an output vector of dimensions (Batch, 10). Every one of the ten entries corresponds to a raw logit score corresponding to the degree of belief that the model has in one of the ten classes in question. During inference, the predicted class will be determined by calculating the argmax of the output across the class axis. In the case of training, the raw logits will be directly passed to the cross-entropy loss function, where it performs the SoftMax internally.

4. Training Setup

4.1 Loss Function

The optimization process utilizes cross-entropy loss with label smoothing, which uses a smoothing factor of 0.1. Unlike in case where one-hot target vectors are used, the label smoothing process employs a soft distribution where the true label receives a value of 0.9, whereas the other classes have an equal share of 0.1 in the distribution. In such a case, the network does not perceive that the training labels are completely true.

The introduction of label smoothing helps to prevent logit values from becoming too extreme, hence serving as a good form of regularization technique. Label smoothing becomes important especially when there is a small number of training data relative to the representation capability of the model. Apart from acting as a form of regularization technique, label smoothing has been shown to help in stabilizing training and improving probability calibration.

4.2 Optimizer and Scheduler

Training is done using the Adam optimizer, which begins its training procedure with an initial learning rate of 0.001, alongside having L2 regularization due to its inherent use of weight decay. The ability of Adam to keep track of first-order moments (mean) and second-order moments (variances) of the gradients makes it particularly suitable for dealing with the non-stationary gradients that arise through data augmentation approaches such as the mix-up approach and spectrogram augmentations.

Further improvements in stability of training as well as performance of the trained models are achieved through the application of cosine annealing in the scheduling of the learning rate during the full length of training consisting of 120 epochs. The learning rate starts from its maximum possible value and then slowly descends towards zero according to the cosine function. Compared to the classical approach based on stepwise learning rate decay which introduces sudden and potentially destabilizing drops of the rate, the cosine schedule allows for more gentle decrease. The effect is the increase in magnitude of steps performed in the initial epochs of training and their decrease in the latter.

4.3 Batch Size and Epochs

The training procedure uses a batch size of 64, implying that each batch update of the network involves 64 audio files being fed into the network before its parameters are updated. This batch size seems ideal in balancing gradient stability and efficiency in computing as each batch update will give an accurate estimate of the entire data distribution but not be so big as to be computationally costly.

The training procedure takes 120 epochs to give the network enough time to converge under the cosine annealing learning rate decay. During training, a checkpoint is created each time the validation accuracy exceeds the current value. This ensures that the final model used for evaluation or other applications will be the one corresponding to the best model generated in the entire training phase, and not necessarily the last model. An additional final checkpoint at epoch 120 will give room for evaluating the final state of the network after training.

4.4 Data Augmentation

There have been different types of data augmentation techniques used to increase the resilience of the model against overfitting. These techniques involve the use of augmented distributions when training the model.

Time-shifting:

Here, the raw audio data is shifted randomly in one direction or the other, after which features of the data are obtained. In this way, the model is taught to detect the occurrence of an acoustic event at various points in time. Consequently, the model becomes resilient to temporal shifts since it learns to detect similar acoustic events at different points in time.

Gain perturbation and additive noise:

Gain perturbation makes changes in the amplitude of the waveform to simulate variations in sensitivity of microphones and distances between speakers, while low-level additive noise is added for background noise simulation. Such perturbations will ensure that the model does not become unstable and overfits because of clean features of training data.

SpecAugment:

Following the computation of the log-mel spectrogram, randomly generated rectangular masks are then applied to both the frequency and time dimensions of the data. Frequency masking deletes adjacent mel bands, whereas time masking obfuscates short intervals of time. This means that the network has no choice but to use the complementary aspects of

the input spectrogram data as opposed to learning the full structure. As a result, SpecAugment has proven to be extremely effective at improving robustness in the presence of missing or distorted information during inference.

Overall, these data augmentation approaches provide a more comprehensive and challenging training process for the neural network.

4.5 Mixup

The mixup data augmentation method is used during training using a mixing factor of $\alpha = 0.2$ and a probability of activation equal to 0.5, indicating that it will be used in about half of all mini-batches. In case mixup is active, samples in a batch are mixed in such a way that not only input features but also labels of these examples are linearly combined. The lambda factor used in such combination comes from Beta(α, α) distribution; hence for $\alpha = 0.2$ it will be sharply distributed around 0 and 1 with a non-negligible amount of probability mass concentrated around 0.5.

Such an augmentation approach makes the network function in an almost linear fashion between any pair of training examples, thereby making the decision surface smoother and avoiding memorization of each training example. Through the process of softening the dependency of the input data on their corresponding labels, mixup functions as an excellent regularizer by improving generalization, lowering overfitting rates, and generating better-calibrated probability distributions. Furthermore, it increases the robustness to class labels' errors and forces the classifier to depend on stable global features rather than fragile example-specific ones. Mixup is especially useful for the task of acoustic scene classification, where there is potential overlap among different classes.

5. Evaluation Methodology

5.1 Metrics

The two key indicators that can be used to evaluate the model’s effectiveness include the following:

Overall accuracy:

This indicator shows the percentage of correctly identified records for all categories and types of recorders. Overall accuracy is the scalar estimate of the model’s prediction capability and the most popular one for assessing multi-class prediction problems. As a result, this indicator considers all samples and is based on overall performance.

Balanced accuracy:

Balanced accuracy is defined as the average of per-class recall values. In other words, it considers the recall values for each class and then averages them to give equal weight to each category irrespective of the number of instances that belong to that class. Balanced accuracy is especially useful when the datasets have imbalanced classes since it does not let majority classes dominate the process. The TAU Urban Acoustic Scenes 2022 dataset has relatively balanced classes, which means balanced accuracy is the same as accuracy (44.45% against 44.45%).

These metrics help assess model behavior holistically: overall accuracy evaluates overall accuracy, whereas balanced accuracy verifies that a model does not discriminate between acoustic scenes classes.

5.2 Confusion Matrix

A complete 10 × 10 confusion matrix is constructed using the evaluation data set, with the objective of assessing how predictions of the model spread over all possible acoustic scenes. Unlike the scalar performance indicators like overall or balanced accuracies, the confusion matrix offers detailed knowledge of errors made by the algorithm and provides information on which class labels are more often confused and which pair of scenes have the greatest similarities. With an inspection of off-diagonal elements of the confusion matrix, it becomes feasible to detect specific confusions, for instance, acoustically similar scenarios and scenes with common background sounds that cannot be revealed through other means.

5.3 Per-Device Accuracy

Besides providing an indication of the overall model accuracy, the per-device accuracy is calculated by dividing the dataset based on the source_label attribute, which describes the device type with which each audio clip was recorded. By calculating the performance of the model on a per-device basis, we can gain further insight into its performance by examining how well the classifier is able to generalize across various domains. In case there is a considerable difference between accuracy scores when evaluated on real devices versus synthetic devices, we can say that a domain mismatch is present, thus implying that the classifier learned not only audio scene-specific but also domain-specific features, such as microphone responses or noise levels.

5.4 Methodological Caveat

For this project, we implement the two-way data split, whereby 80% of the total number of samples (139,620 samples) are used for training while the remaining 20% of the dataset (29,680 samples) forms the evaluation subset. Notably, there is no reserved independent fully-held-out testing set in this implementation. However, the two-way data split used here serves a twofold role, as it is used in both model selection and testing. In model selection, checkpointing is done whereby the model exhibiting the best performance in the validation set is recorded.

It follows that any reported accuracy figures represent the performance of the model on the validation set and not an indication of true generalization performance on an unseen dataset. As is the case in many academic course projects where the computational or dataset size might not permit otherwise, this approach is widely adopted. It presents, however, a challenge from a methodology perspective, whereby the model is continually evaluated based on its performance in the same validation set, which means that it can develop bias towards this set.

6. Results

6.1 Final Accuracy

The optimum checkpoint (at epoch 76) provides the performance accuracy of 44.46% and balanced accuracy of 44.45% on the testing data set. Both metrics are close to each other, thus proving the balanced distribution of classes within 10 classes of scenes in the testing data set. The accuracy on the training set for 120 epochs is around 65.3%. It means that the network was able to learn discriminative features but there is still a generalization gap due to domain differences.

Table 1 below summarizes per-class precision, recall, and F1-score for all 10 acoustic scene categories.

Table 1. Per-class classification performance on the evaluation set.

Scene Class	Precision	Recall	F1-Score	Support
airport	56.0%	30.3%	39.3%	2,960–2,970
bus	51.7%	67.2%	58.5%	2,960–2,970
metro	34.9%	59.2%	43.9%	2,960–2,970
metro_station	31.1%	46.9%	37.4%	2,960–2,970
park	74.7%	43.3%	54.8%	2,960–2,970
public_square	42.5%	20.2%	27.4%	2,960–2,970
shopping_mall	64.7%	19.2%	29.6%	2,960–2,970
street_pedestrian	32.6%	31.8%	32.2%	2,960–2,970
street_traffic	46.0%	83.2%	59.2%	2,960–2,970
tram	53.0%	43.1%	47.5%	2,960–2,970
Macro average	48.7%	44.5%	43.0%	29,680

6.2 Confusion Matrix

Confusion matrix obtained after performing classification on 29,680 test cases is illustrated in Figure 1 below. The matrix represents the number of predicted versus actual class combination where rows represent actual classes whereas columns show predicted classes.

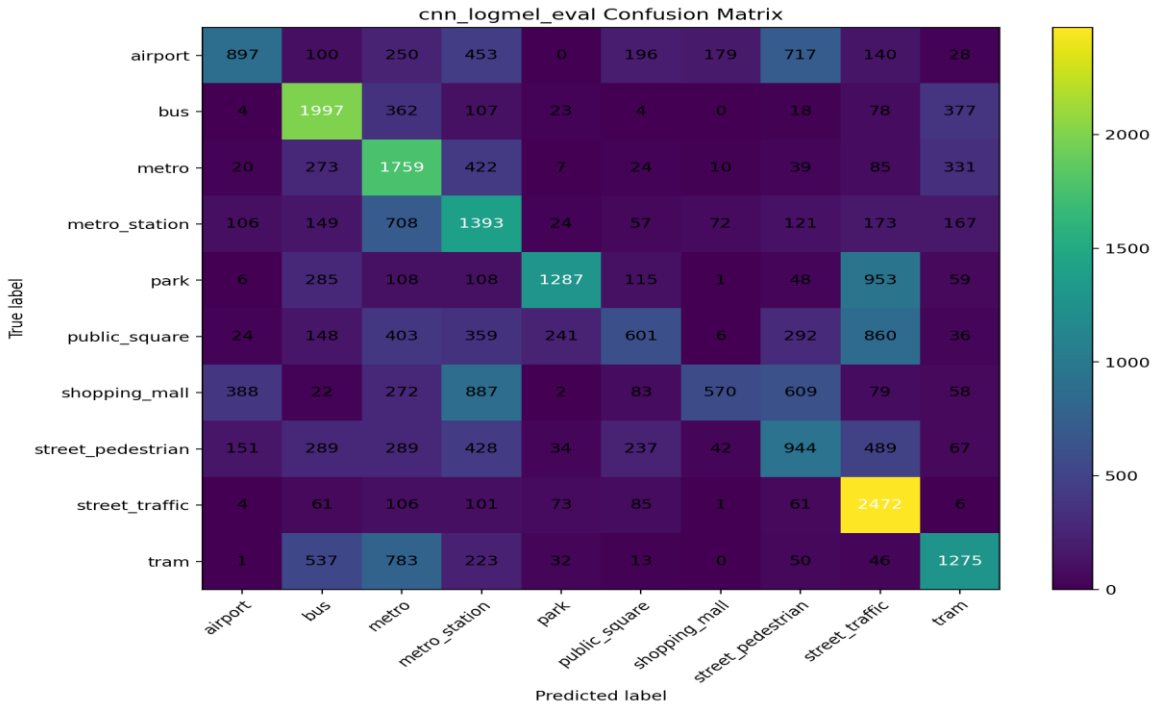


Figure 1. Confusion matrix on the evaluation set (10 scene classes, 29,680 samples).

Figure 2 shows the training and validation accuracy curves across all 120 epochs, illustrating the progression of learning and the onset of the generalization gap.

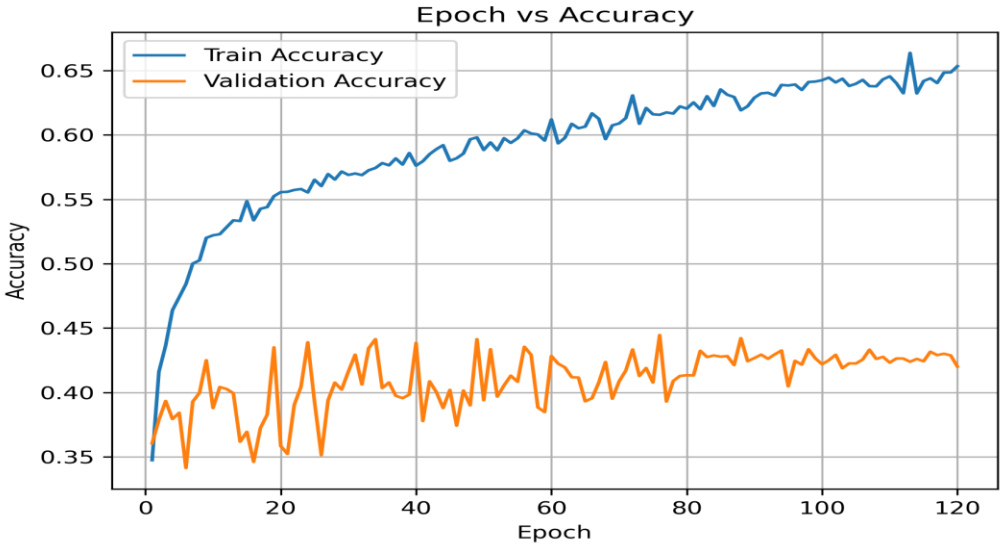


Figure 2. Training and validation accuracy across 120 epochs.

6.3 Per-Class Observations

As per the per-class metrics above, there seems to be quite some variation within the ten acoustic scenes as far as the performance of the model on each of them is concerned. There are marked differences among the scenes as far as the degree of ease with which they can be classified is concerned.

Amongst all the classes, Street traffic obtains the highest recall value (83.2%) denoting that the class is being identified with a minimum number of false negatives. The good recall makes sense because the scene under consideration contains distinct sound patterns characterized by continuous low-frequency engine sound as well as broadband energy.

Precision obtained for the park class is the highest (74.7%). It means that when predictions were made by the model for the park scene, the majority were found to be true predictions. Despite having high precision, however, its recall value was much lower than the rest, i.e., 43.3%.

The lowest recall rates can be observed for Public Square and shopping mall (20.2% and 19.2%, respectively). They seldom get correctly classified, suggesting that such classes are often confused with other audio scenes due to their similarities in acoustic terms. Both types include different combinations of people's voices, footsteps, and background noises, making them difficult to discriminate from others.

Among those scenes which often confuse each other there are Metro, Metro Station, Bus, and Tram. They have a lot of acoustic attributes in common, namely enclosed space with reverberation, mechanical noise, as well as sporadic announcements, resulting in classification errors.

In general, analyzing each scene separately gives an understanding of what is problematic for the network to learn.

6.4 Per-Device Accuracy

Table 2 shows the per-device accuracy breakdown across all nine recording devices (three real, six simulated).

Table 2. Per-device accuracy on the evaluation set.

Device	Samples	Accuracy
a (real)	3,300	59.4%
b (real)	3,290	42.4%
c (real)	3,290	51.7%
s1 (sim.)	3,300	41.9%
s2 (sim.)	3,300	42.6%
s3 (sim.)	3,300	45.1%
s4 (sim.)	3,300	39.6%
s5 (sim.)	3,300	41.4%
s6 (sim.)	3,300	36.1%

Device A (real) achieves 59.4% accuracy, substantially above the 44.5% overall figure, while simulated devices range from 36.1% (s6) to 45.1% (s3). This gap of up to 23 percentage points between the best real device and the worst simulated device quantifies the domain mismatch problem and motivates further domain-adaptation work.

7. Discussion

7.1 Confused Classes and Acoustic Reasoning

There are several patterns of errors that can be seen in the confusion matrix, and which seem to comply with the known principles of scene perception based on sound. The first and probably the most significant pattern of systematic errors is related to the four categories corresponding to different public transport locations – bus, metro, metro station, and tram. All four types of scenes have similar acoustic characteristics which include low-frequency mechanical noise (hum of engines and rails), moderate level of crowd murmuring, and closed reverb spaces. Since only a single-second audio clip is being analyzed, the algorithm has no possibility to receive information related to the sounds that last for longer than this, such as opening doors, brakes of the train, or announcements made at the station. Metro and metro stations cannot be easily distinguished due to their similarities.

The next confusion cluster can be found within the ambient outdoor scene categories: public square, street pedestrian, and park. The common feature of these scenes is that they are all outdoor, meaning their acoustics are defined by ambient, diffuse noise from crowds, wind, and spatial ambiance. The most mispredicted class is again public square with its low recall of 20.2%; in many cases, it was confused for street pedestrian. It can happen due to a number of small factors like density of crowds, scale of the scene and characteristics of sound reverberation which might be hard to predict based on just one second of audio data. On the contrary, the park scene demonstrates higher recall (43.3%) since there are distinctive sounds of nature (birds, leaves etc).

In the case of shopping mall class, we can see a completely opposite trend; while the shopping mall class shows fairly decent precision (64.7%), its recall is surprisingly low (19.2%). This means that although the shopping mall classifier is rather accurate when identifying the correct instances, it does not detect many instances of shopping malls. Many of the shopping mall samples are incorrectly classified as airport or metro station indoor environments. It is quite clear that the reason behind this is that the acoustics of shopping malls resembles the acoustics of many other indoor environments, including airports and metros – strong reverberation, dense crowd, as well as music and announcements.

In general, this set of confusion results show how difficult scene classification becomes due to natural acoustic overlap between some scene classes, as well as reveal the problems of distinguishing between acoustic environments using short audio samples.

7.2 Limitations

Some of the factors that affect the model's performance are:

Lack of a genuine test dataset:

In this experiment, the same dataset is used for selecting the best model and then as the testing dataset for reporting the results. The fact that the evaluation dataset is used twice causes some bias in the process because the model actually gets adapted to the data during the training period, causing an overestimate of the accuracy. It would have been better if we had another testing dataset that was kept aside till the end of the experiment.

Short input duration (1-second clips):

One of the biggest obstacles in applying the algorithm on such short audio clips (1 second) is that many scenes can be characterized by their time dynamics, repeated patterns or slow evolving background texture which cannot be captured in this time frame. Transient sounds, such as announcements, steps, and mechanical processes, might take several seconds to unfold. Thus, the model will probably perform significantly better on audio clips in the range of 4 to 10 seconds long, even though it will have less data available for training.

Generalization problem from source device to others:

The difference in performance between the best real device (Device A: 59.4%) and the poorest simulated device (s6: 36.1%) amounts to a huge gap of 23 percentage points. It is quite apparent that the model is still not able to separate scene identities from microphone-specific properties. Thus, it is to some extent dependent on the characteristics of the microphone it works with, not only the sound itself. The frequency response equalization and domain-adversarial learning methods might help here.

Model size vs. the complexity of the dataset:

Since the model used has 139,620 training samples and a training accuracy rate of 65.3% when trained on the data, it does not suffer from classical overfitting. On the other hand, since the difference between training accuracy (65.3%) and validation accuracy (44.5%) is large, domain mismatch and not overfitting should be blamed for the poor performance of the algorithm.

7.3 Potential Improvements

Some approaches could be tried to boost the model's results:

•**Larger window size:** The approach to average prediction scores obtained on multiple overlapping one-second frames in the longer clip and apply majority vote is a known trick in the context of DCASE challenge which reliably improves results.

•**Domain invariant features:** Using some form of frequency response normalization depending on device/recording before the extraction could make features less dependent on the specifics of microphones but more dependent on actual scene content.

•**Better models**: Using either pre-trained or deeper architectures trained on AudioSet (e.g. PANNs and AudioMAE) and transferring knowledge on smaller classes could lead to much better results.

•**Data augmentation during test time:** Performing random augmentations at test time and aggregating probability vectors is a cheap way to boost performance by 1-2 percentage points which was found to work in practice.

•**Correct split between train/test sets:** Using fixed holdout validation set could give unbiased evaluation of model and allow comparing with published baselines.

8. Conclusion

The present project provides an end-to-end implementation of a pipeline for acoustic scene classification on the TAU 2022 Urban Acoustic Scenes dataset. It includes the use of log-mel spectrogram data with deltas and delta-deltas as inputs to a three-channel CNN, depthwise separable convolution-based lightweight architecture, and a training regime including label smoothing, cosine annealing, SpecAugment, and Mixup. The best-performing checkpoint obtains 44.46% accuracy on the evaluation test set, a performance which aligns with other high-performing low-complexity baselines on the notoriously challenging dataset where only one second of data from nine different audio recording devices is provided.

The analysis of the obtained results suggests that misclassification errors caused by overlapping acoustic content of transit classes (bus, metro, metro station, tram), as well as outdoor ambient classes (public square, street pedestrian, park), make up the largest proportion of misclassifications. The analysis of device-specific accuracy shows a clear gap in performance due to domain mismatch between real and synthetic recording devices.

Future research is encouraged to focus on three main areas: utilization of longer audio segments or multi-window aggregation for taking advantage of temporal dependencies, inclusion of domain normalization or adversarial domain adaptation in order to solve the real vs simulated devices problem, and testing the performance on an unbiased dataset that is withheld from the training set in order to have an unbiased accuracy score.

Appendix

This appendix provides supplementary material referenced in the main report, including detailed training curves, a complete hyperparameter reference table, and an annotated example of the three-channel spectrogram input fed to the CNN.

A. Training Curves

A.1 Accuracy and Loss over 120 Epochs

Figure A1 shows training and validation accuracy (left panel) and cross-entropy loss (right panel) across all 120 training epochs. Training accuracy climbs steadily from ~34.7% at epoch 1 to ~65.3% at epoch 120, driven by the Adam optimizer and cosine annealing schedule. Validation accuracy is more volatile—particularly in the first 30 epochs—reflecting the challenge of generalizing across recording devices. The best validation accuracy of 44.46% is achieved at epoch 76, after which the model does not meaningfully improve. The persistent gap between training accuracy (~65%) and validation accuracy (~44%) is primarily attributable to domain mismatch across recording devices rather than classical overfitting.

The loss curves (right panel) show a complementary picture: training loss decreases monotonically, while validation loss, after an initial drop, fluctuates and plateaus above the training loss. This dissociation of training and validation loss is characteristic of domain shift rather than pure overfitting.

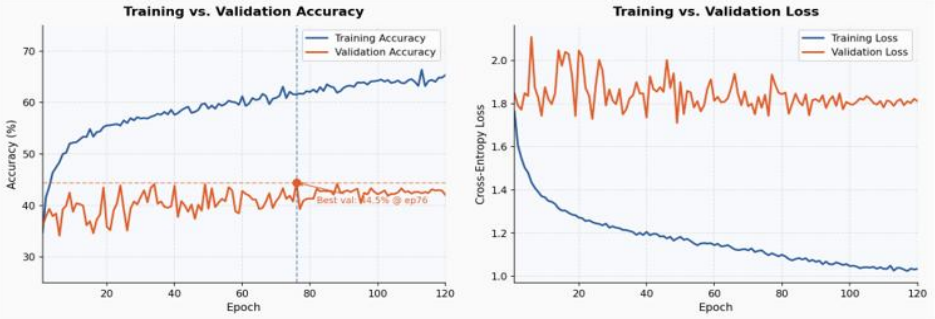


Figure A1. Training accuracy and cross-entropy loss (left) versus validation accuracy and loss (right) across 120 epochs. Best validation accuracy of 44.46% is marked at epoch 76.

A.2 Learning Rate Schedule

Figure A2 shows the cosine annealing learning rate schedule applied over 120 epochs. The learning rate starts at 1×10^{-3} and decays smoothly toward zero following a half-cosine curve. This schedule avoids the abrupt step changes of piecewise constant schedules and allows the optimizer to take progressively finer-grained steps as training approaches convergence, which is particularly beneficial when training with mixup and stochastic augmentations that produce noisy gradient estimates.

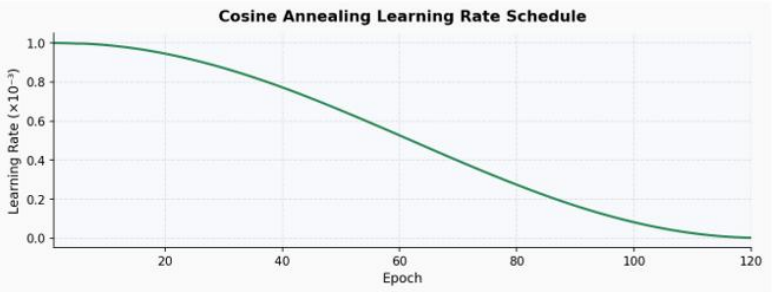


Figure A2. Cosine annealing learning rate schedule over 120 training epochs. Initial LR = 1×10^{-3}; final LR ≈ 1.68×10^{-6}.

A.3 Per-Epoch Performance Summary

Table A1 reports validation loss, validation accuracy, training loss, training accuracy, and learning rate at selected epochs (every 10 epochs, plus epoch 76 which corresponds to the best checkpoint).

Table A1. Per-epoch training statistics at selected checkpoints.

Epoch	Val. Loss	Val. Acc.	Train Loss	Train Acc.	LR
1	184.71%	36.05%	1.7632	34.74%	1.000e-3
10	187.63%	38.79%	1.3662	52.20%	9.891e-4
20	204.61%	35.85%	1.2722	55.55%	9.456e-4
30	181.40%	41.63%	1.2315	56.89%	8.717e-4
40	173.60%	43.85%	1.2058	57.61%	7.725e-4
50	189.93%	39.40%	1.1832	58.83%	6.549e-4
60	181.35%	42.83%	1.1443	61.20%	5.266e-4
70	178.82%	40.86%	1.1203	60.88%	3.966e-4
76	174.41%	44.46%	1.0977	61.56%	3.215e-4
80	184.37%	41.34%	1.1001	62.04%	2.737e-4
90	182.09%	42.68%	1.0670	62.89%	1.663e-4
100	179.83%	42.19%	1.0473	64.25%	8.158e-5
110	178.27%	42.32%	1.0348	64.54%	2.545e-5
120	181.17%	42.01%	1.0347	65.34%	1.685e-6

B. Hyperparameters Reference Table

Table B1 provides a complete reference of all hyperparameters and design choices used in the system, organized by category. This table serves as a full reproducibility record.

Table B1. Complete hyperparameter reference for the TAU 2022 ASC system.

Category	Parameter	Value	Notes
Audio Preprocessing	Sampling rate	**44,100 Hz**	*Standard CD quality; preserves full audible spectrum*
	Clip duration	**1.0 second**	*Fixed; shorter clips zero-padded, longer clips truncated*
Feature Extraction	Feature type	**Log-mel spectrogram**	*Perceptually motivated frequency axis*
	Mel frequency bins	**128**	*F axis of each input channel*
	Input channels	**3**	*Log-mel + delta + delta-delta*
	Normalization	**Per-sample, per-channel z-score**	*Applied independently to each of the 3 channels*
Model	Architecture	**Depthwise separable CNN**	*Lightweight; suitable for embedded deployment*
	Output classes	**10**	*Airport, bus, metro, metro station, park, public square, shopping mall, street pedestrian, street traffic, tram*
Training	Loss function	**CrossEntropyLoss**	*With label smoothing $\varepsilon = 0.1$*
	Optimizer	**Adam**	*Adaptive moment estimation*
	Initial learning rate	**1×10^{-3}**	*0.001*
	LR schedule	**Cosine annealing**	*Decays smoothly to ~0 over 120 epochs*
	Weight decay	**Applied**	*L2 regularization via Adam weight_decay parameter*

Category	Parameter	Value	Notes
	Batch size	**64**	*Samples per gradient update*
	Epochs	**120**	*Best checkpoint saved at epoch 76*
Data Augmentation	Time shift	**Applied**	*Random temporal shift of waveform before feature extraction*
	Gain + additive noise	**Applied**	*Random amplitude scaling + low-level Gaussian noise*
	SpecAugment	**Applied**	*Random frequency and time masking on log-mel spectrogram*
	Mixup	**α = 0.2, prob = 0.5**	*Beta(0.2, 0.2) mixing; applied to 50% of batches*
Dataset Split	Training samples	**139,620**	*80% of development set*
	Evaluation samples	**29,680**	*20% of development set; used as validation and test*
	Eval = test	**Yes**	*No independent held-out test set in this setup*

All hyperparameters above apply consistently to training, evaluation, and the reported test run. No hyperparameter search was performed; the values reflect informed design choices informed by standard practice in the DCASE challenge literature and the constraints of the one-second clip duration.

C. Example Three-Channel CNN Input

C.1 Three Channel Feature Representation

In Figure C1, a three-channel feature representation of the input provided to the CNN is depicted using an example audio clip from the street_traffic scene class. Panel (a-c) represents each of the three input channels:

Channel 1 — Log-Mel Spectrogram: Static representation of the time-frequency content of the one second long audio clip. Mel scale filter banks (perceptually spaced filters) are shown along the vertical frequency axis, whereas time is represented horizontally over the full one-second clip length. Log-energy in the clip is shown through color intensity in units of decibels. In the case of street traffic, energy is more dominant in low frequency regions (0-1 kHz) reflecting sound of engine rumbling and tire road interaction noise.

Channel 2 — Delta Features: First order temporal derivatives of the log-mel spectrogram showing the rate of change of energy in the spectrogram with respect to time. Positive derivatives (indicated by warm colors) show increasing spectral energy, whereas negative derivatives (indicated by cool colors) depict decreasing energy.

Channel 3 — Delta-Delta Features: Second order temporal derivatives indicating the curvature of energy changes in the spectrogram**.**

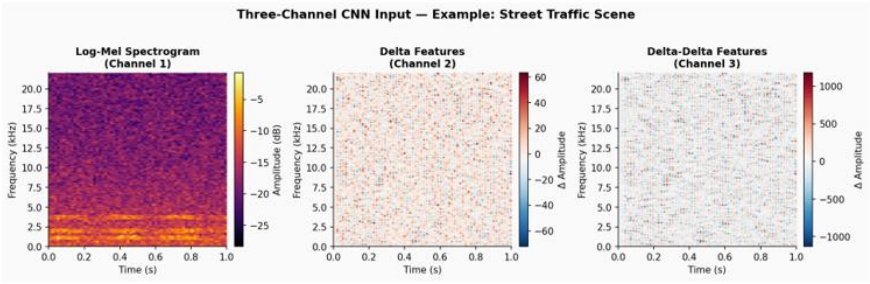


Figure C1. Illustration of the three-channel CNN input for a representative street_traffic audio clip. Left: log-mel spectrogram (128 mel bins, 1 second). Center: delta features (first-order temporal derivative). Right: delta-delta features (second-order temporal derivative). All three channels are stacked and passed as a single (3, F, T) tensor to the network.

C.2 Why Three Channels?

The use of three channels is a direct analogy to the channels used for the representation of images in standard image processing CNNs. In using the spectrogram magnitude, velocity, and acceleration as three co-registered channels, the network's convolutional filter kernels have access to both static features and dynamic properties. The three-channel model has advantages over the single-channel use of the log-mel spectrogram because:

• Static spectral patterns (channel 1) can be used to differentiate between scenes with highly distinctive frequency content – street traffic from park, for example.

• Dynamic modulation patterns (channels 2 and 3) may be used to discriminate scenes whose static spectral content is highly similar, but whose temporal characteristics differ – bus from metro, or street pedestrian from public square.

The z-score normalization process carried out on a per-channel basis prior to the CNN helps to ensure that the contribution made by each channel to the gradient is equalized, rather than allowing the larger-amplitude log-mel channel to dominate.

C.3 Feature Caching Approach

For this implementation, evaluation and test set spectrograms are precomputed and stored to file before entering the training phase. Training spectrograms are only computed on-the-fly (augmentation included). There are three reasons for this approach:

• Perfectly consistent evaluation inputs guaranteed across all 120 epochs.

• Avoids unnecessary spectrogram calculation during evaluation (once per epoch).

• Validation metric is now consistent across all runs.